Linux Network Driver Architecture & TX/RX Ring Initialization — A Practical Guide

Author: M365 Copilot

Audience: Kernel/Driver developers and advanced systems engineers

Overview

This guide walks you from first principles to a working mental model of how modern Linux Ethernet drivers initialize and operate **TX/RX descriptor rings**. It builds on the standard net_device and NAPI architecture and shows step-by-step how to allocate coherent descriptor memory, map/unmap packet buffers with the DMA API, manage producer/consumer indices, and wire things into the open/stop and NAPI poll paths. The patterns here apply across PCIe NICs and SoC MACs, with placeholders for device-specific doorbells and ownership bits.

What You'll Learn

- Linux netdev building blocks: net_device, net_device_ops, NAPI, ethtool, and statistics.
- How descriptor rings work: coherent descriptor memory, streaming mappings for data buffers, and ownership.
- RX ring initialization: pre-posting buffers, publishing the tail, and refill strategies.
- TX ring initialization and transmit: space checks, mapping, doorbells, and completion handling.
- How NAPI coordinates RX/TX completions and budgets.
- Open/stop, IRQs/MSI-X, multiqueue scaling, and common pitfalls & debugging.

Assumptions & Versions

Examples target Linux 5.10+ APIs (valid for 5.15/6.x with minor tweaks). Replace device-specific MMIO/doorbell details with your NIC's registers. For platform devices, use the platform "struct device" instead of PCI's.

1) Linux netdev building blocks (crash course)

A Linux Ethernet driver centers around a "struct net_device". You allocate it with "alloc_netdev()", initialize fields in a setup function (often calling "ether_setup()"), and register it with "register_netdev()". Runtime operations are provided via "net_device_ops" (open/stop/start_xmit/etc.) and optionally "ethtool_ops" for introspection and tuning. RX is typically driven by interrupts but delivered under NAPI polling to bound CPU overhead under load.

```
static const struct net_device_ops vnet_netdev_ops = {
    .ndo_open        = vnet_open,
    .ndo_stop        = vnet_stop,
    .ndo_start_xmit  = vnet_start_xmit,
    .ndo_get_stats64 = vnet_get_stats64,
};

static void vnet_setup(struct net_device *dev)
{
    ether_setup(dev);
    dev->netdev_ops = &vnet_netdev_ops;
    dev->min_mtu = 68;
    dev->max_mtu = ETH_DATA_LEN;
}
```

NAPI: Register with "netif_napi_add()" and implement "poll()" to consume work up to a budget. Schedule NAPI from your ISR on RX/TX events; complete the poll with "napi_complete_done()" and re-enable IRQs.

```
// Register NAPI during device init
netif_napi_add(dev, &priv->napi, vnet_poll, 64);

// Enable in ndo_open(), disable in ndo_stop()
napi_enable(&priv->napi);
napi_disable(&priv->napi);
```

2) Descriptor rings & DMA (how the hardware sees memory)

A descriptor ring is a circular array describing packet buffers. The **descriptor ring** itself is usually allocated with "dma_alloc_coherent()" so both CPU and device share a consistent view. Actual packet data buffers are mapped per-packet using streaming mappings ("dma_map_single()"/"dma_unmap_single()"), in the direction of transfer: **DMA_FROM_DEVICE** for RX, **DMA_TO_DEVICE** for TX.

```
#define RING_SIZE      1024            // power of two simplifies wrap
#define RING_MASK      (RING_SIZE - 1)
#define RX_BUF_SIZE    2048

struct vnet_desc {
    __le64 addr;   // DMA address of data buffer
    __le32 len;    // buffer size (RX) or payload (TX)
    __le16 flags;  // device-specific: OWN/EOP/RS/etc.
    __le16 csum;   // optional / reserved
} __packed;

struct vnet_txbuf {
    struct sk_buff *skb;
    dma_addr_t dma;
    u16 len;
    bool mapped;
};
```

```
struct vnet_rxbuf {
    struct sk_buff *skb;
    dma_addr_t dma;
    u16 len;
    bool posted;
};

struct vnet_ring {
    struct vnet_desc *desc; // CPU VA
    dma_addr_t desc_dma;    // bus address seen by device
    size_t bytes;           // ring bytes

    u16 head;   // software consumer
    u16 tail;   // software producer
    u16 size;   // RING_SIZE

    union { struct vnet_txbuf *tx; struct vnet_rxbuf *rx; void *any; } buf;

    void __iomem *dbell;    // device doorbell register (tail)
    void __iomem *hw_head;  // optional: device head pointer
};
```

Indices & ownership: The driver advances **tail** when posting work to the device. The device advances a hardware head pointer as it completes entries; the driver mirrors this into its software **head**. A power-of-two size and a mask simplify wrapping: "(idx + 1) & RING_MASK".

3) RX ring initialization (pre-posting buffers)

```
static int ring_alloc(struct device *dma_dev, struct vnet_ring *r, bool tx)
{
    r->size  = RING_SIZE;
    r->bytes = r->size * sizeof(struct vnet_desc);
    r->desc  = dma_alloc_coherent(dma_dev, r->bytes, &r->desc_dma, GFP_KERNEL);
    if (!r->desc)
        return -ENOMEM;

    memset(r->desc, 0, r->bytes);
    r->head = r->tail = 0;

    if (tx) {
        r->buf.tx = kzalloc(r->size * sizeof(*r->buf.tx), GFP_KERNEL);
        if (!r->buf.tx) goto err;
    } else {
        r->buf.rx = kzalloc(r->size * sizeof(*r->buf.rx), GFP_KERNEL);
        if (!r->buf.rx) goto err;
    }
    return 0;
err:
    dma_free_coherent(dma_dev, r->bytes, r->desc, r->desc_dma);
    r->desc = NULL;
    return -ENOMEM;
}

static int rx_alloc_one(struct device *dma_dev, struct net_device *ndev,
                        struct vnet_ring *rxr, u16 i)
{
    struct vnet_rxbuf *b = &rxr->buf.rx[i];
    struct sk_buff *skb = netdev_alloc_skb_ip_align(ndev, RX_BUF_SIZE);
```

```
    dma_addr_t dma;

    if (!skb)
        return -ENOMEM;

    dma = dma_map_single(dma_dev, skb->data, RX_BUF_SIZE, DMA_FROM_DEVICE);
    if (dma_mapping_error(dma_dev, dma)) {
        dev_kfree_skb_any(skb);
        return -EIO;
    }

    b->skb = skb; b->dma = dma; b->len = RX_BUF_SIZE; b->posted = true;
    rxr->desc[i].addr  = cpu_to_le64(dma);
    rxr->desc[i].len   = cpu_to_le32(b->len);
    rxr->desc[i].flags = cpu_to_le16(0); // device-specific OWN
    return 0;
}

static int rx_ring_fill(struct device *dma_dev, struct net_device *ndev, struct vnet_ring *rxr)
{
    for (u16 i = 0; i < rxr->size; i++) {
        int rc = rx_alloc_one(dma_dev, ndev, rxr, i);
        if (rc) return rc;
    }
    // Publish tail to hardware (device-specific):
    // writel(rxr->size - 1, rxr->dbell);
    rxr->tail = rxr->size; // SW book-keeping
    return 0;
}
```

Refill policy: Keep the RX ring near full. Either re-post immediately after consuming an entry in NAPI, or batch refills when the free count drops below a threshold to reduce MMIO traffic.

4) TX ring initialization & transmit path

```
static void tx_ring_init(struct vnet_ring *txr)
{
    memset(txr->desc, 0, txr->bytes);
    memset(txr->buf.tx, 0, txr->size * sizeof(*txr->buf.tx));
    txr->head = txr->tail = 0;
}

static netdev_tx_t vstart_xmit(struct sk_buff *skb, struct net_device *ndev)
{
    struct vnet_priv *priv = netdev_priv(ndev);
    struct vnet_ring *txr = &priv->txr;
    u16 i = txr->tail;

    if (((i + 1) & RING_MASK) == txr->head) {
        netif_stop_queue(ndev);
        return NETDEV_TX_BUSY; // no space
    }

    // Map only linear data for brevity (handle frags in production)
    unsigned int len = skb_headlen(skb);
    dma_addr_t dma = dma_map_single(priv->dma_dev, skb->data, len, DMA_TO_DEVICE);
    if (dma_mapping_error(priv->dma_dev, dma)) {
        dev_kfree_skb_any(skb);
```

```
            return NETDEV_TX_OK;
        }

        txr->desc[i].addr  = cpu_to_le64(dma);
        txr->desc[i].len   = cpu_to_le32(len);
        txr->desc[i].flags = cpu_to_le16(/* EOP | RS | OWN */ 0);

        txr->buf.tx[i].skb = skb;
        txr->buf.tx[i].dma = dma;
        txr->buf.tx[i].len = len;
        txr->buf.tx[i].mapped = true;

        // Advance producer and ring doorbell
        txr->tail = (i + 1) & RING_MASK;
        // writel(txr->tail, txr->dbell);

        netdev_tx_sent_queue(netdev_get_tx_queue(ndev, 0), len);
        return NETDEV_TX_OK;
    }
```

TX completion: In your NAPI poll or ISR, read the hardware head (or status bits) to learn which descriptors the NIC has consumed. For each completed entry, unmap the DMA buffer and free/consume the SKB. If the queue was stopped due to ring full, call "netif_wake_queue()" when enough space becomes available.

5) NAPI poll: combining RX delivery and TX completion

```
    static int vnet_poll(struct napi_struct *napi, int budget)
    {
        struct vnet_priv *priv = container_of(napi, struct vnet_priv, napi);
        struct vnet_ring *txr = &priv->txr;
        struct vnet_ring *rxr = &priv->rxr;
        int work_done = 0;

        // --- TX completions (device-specific head read omitted) ---
        while (/* txr->head != hw_tx_head */ false) {
            u16 i = txr->head;
            if (txr->buf.tx[i].mapped) {
                dma_unmap_single(priv->dma_dev, txr->buf.tx[i].dma,
                                 txr->buf.tx[i].len, DMA_TO_DEVICE);
                txr->buf.tx[i].mapped = false;
            }
            if (txr->buf.tx[i].skb) {
                dev_consume_skb_any(txr->buf.tx[i].skb);
                txr->buf.tx[i].skb = NULL;
            }
            txr->head = (txr->head + 1) & RING_MASK;
        }

        // --- RX completions ---
        while (work_done < budget) {
            if (/* no RX ready */ true)
                break;

            u16 i = rxr->head;
            struct vnet_rxbuf *b = &rxr->buf.rx[i];
            struct sk_buff *skb = b->skb;
            u16 pkt_len = /* device provided length */ 64;
```

```
            dma_unmap_single(priv->dma_dev, b->dma, b->len, DMA_FROM_DEVICE);

            skb_put(skb, pkt_len);
            skb->protocol = eth_type_trans(skb, priv->dev);
            skb->ip_summed = CHECKSUM_UNNECESSARY;
            napi_gro_receive(napi, skb);

            // Refill the slot immediately
            if (rx_alloc_one(priv->dma_dev, priv->dev, rxr, i)) {
                b->posted = false; // out of memory: leave a hole
            }

            rxr->head = (rxr->head + 1) & RING_MASK;
            work_done++;
        }

        if (work_done < budget) {
            napi_complete_done(napi, work_done);
            // re-enable device IRQs here
        }

        return work_done;
    }
```

6) Wiring into open()/stop()

```
    static int vnet_open(struct net_device *ndev)
    {
        struct vnet_priv *priv = netdev_priv(ndev);
        int rc;

        napi_enable(&priv->napi);

        rc = ring_alloc(priv->dma_dev, &priv->txr, true);
        if (rc) goto err_napi;
        rc = ring_alloc(priv->dma_dev, &priv->rxr, false);
        if (rc) goto err_free_tx;

        tx_ring_init(&priv->txr);

        rc = rx_ring_fill(priv->dma_dev, ndev, &priv->rxr);
        if (rc) goto err_free_rx;

        // Program device registers (bases/sizes), enable queues & IRQs here

        netif_start_queue(ndev);
        return 0;

    err_free_rx:
        // free RX ring
        // ...
    err_free_tx:
        // free TX ring
        // ...
    err_napi:
        napi_disable(&priv->napi);
        return rc;
    }
```

```
static int vnet_stop(struct net_device *ndev)
{
    struct vnet_priv *priv = netdev_priv(ndev);

    netif_stop_queue(ndev);
    napi_disable(&priv->napi);

    // Disable device queues/IRQs

    // Clean and free TX/RX rings (unmap and free SKBs)
    // ...
    return 0;
}
```

7) Interrupts, doorbells, and memory ordering

Doorbells: After the driver writes descriptors, it rings a device-specific doorbell (often by writing the new tail index to an MMIO register). **Interrupts** indicate completions or RX arrivals; in MSI-X designs, each queue has its own vector. **Memory ordering**: Ensure descriptors are visible to the device before ringing the doorbell ("wmb()" or implicit ordering if device requires). When reading device head pointers, use appropriate barriers ("rmb()").

8) Multiqueue and scaling (RSS, NAPI per-queue)

Modern NICs expose multiple TX/RX queue pairs. Bind each to its own NAPI instance and MSI-X vector for parallelism. Use "netif_set_real_num_tx_queues()" and "netif_set_real_num_rx_queues()" to advertise queue counts; consider XPS (transmit packet steering) and RSS (receive side scaling) to distribute load across CPUs.

9) Common pitfalls & debugging tips

- **DMA mapping errors**: Always check "dma_mapping_error()"; directions must match (RX=FROM_DEVICE, TX=TO_DEVICE).
- **Headroom & alignment**: Use "netdev_alloc_skb_ip_align()" to provide IP alignment. Reserve headroom if your NIC requires extra descriptors or headers.
- **TSO/CSUM offloads**: If you advertise offloads, make sure you program descriptors accordingly, or clear feature flags.
- **Queue stopping**: Stop the TX queue when ring is full and wake on completions; otherwise you'll hit warnings about timeouts.
- **MTU changes**: Recalculate RX buffer size and reinitialize rings on MTU updates (ndo_change_mtu).
- **Barriers**: Ensure "wmb()" before doorbells and "readl()/writel()" ordering per your device's spec.
- **Leak checks**: On stop/unload, unmap and free all SKBs; purge RX rings and consume any stray TX SKBs.

10) Testing & validation

- **Smoke**: Bring link up, assign IP, and verify counters via "ip -s link".
- **ethtool**: Implement "get_drvinfo" and optionally "get_ethtool_stats" to expose counters.

- **Traffic**: Use "pktgen", "iperf3", "tcpreplay", and "tcpdump" on RX to verify deliveries.
- **Tracing**: net/core tracepoints, ftrace function_graph on your driver, and perf to observe NAPI cycles.

11) Minimal loopback virtual driver (for context)

If you're not connecting to real hardware yet, you can simulate RX using an internal queue (e.g., "skb_queue"). The earlier "vnet_sample" skeleton looped TX back into RX via NAPI. When you're ready for real NIC work, replace the queue with descriptor rings as shown in sections 3–5.

```
// In a pure-virtual sample, you can avoid DMA and rings:
// 1) Clone TX skb and enqueue to a SW RX queue
// 2) Schedule NAPI
// 3) In poll(), dequeue and deliver via napi_gro_receive()
```

Appendix: Handy APIs & macros

```
Allocation & registration:
  alloc_netdev(), register_netdev(), free_netdev()
NAPI:
  netif_napi_add(), napi_schedule(), napi_complete_done()
DMA mapping:
  dma_alloc_coherent(), dma_free_coherent()
  dma_map_single(), dma_unmap_single(), dma_mapping_error()
SKBs:
  netdev_alloc_skb_ip_align(), skb_put(), skb_pull(), skb_reset_mac_header()
Protocol setup:
  eth_type_trans(), ether_setup(), eth_hw_addr_random()
Queuing:
  netif_start_queue(), netif_stop_queue(), netif_wake_queue()
Stats:
  ndo_get_stats64(), u64_stats_sync
Barriers & MMIO:
  wmb(), rmb(), readl(), writel()
```

Notes

This guide provides device-agnostic ring logic. For a real NIC, consult the vendor's datasheet for descriptor formats, doorbell semantics, ownership bits, and interrupt moderation registers. Adapt the RX buffer size to MTU and offloads (e.g., 9K jumbo frames).